

Lexical Analyzer

Kedar Sedai

Abstract—MiniLang is a small research compiler for a minimal imperative language, implemented in C++17 as a classic multi-phase pipeline: *lexical analysis, parsing, semantic analysis, intermediate representation, code optimization and LLVM-based code generation*. The current implementation provides a hand-written lexical analyzer that tokenizes source with accurate line/columns locations, keyword recognition, comment handling and a command line lexer driver for testing.

I. INTRODUCTION

This Lexical analyzer(lexer) is the first stage of the compiler. It reads source text and produces a sequence of **tokens** the smallest meaningful units of the language-each tagged with a kind, source spelling, and location. The lexer performs a single left-to-right pass over an in-memory source buffer. Comments and whitespace are not emitted as tokens.

Item	Details
Implementation	Hand-written scanner in C++17 (no Flex/Lex)
Namespace	miniLang
Headers	include/minilang/lexer/lexer.cpp, token.hpp
Sources	src/lexer/lexer.cpp, token.cpp
Driver	minilang_lexer(src/tools/lexer_main.cpp)

TABLE I: Overview

II. ARCHITECTURE

Source file -> read into std::string -> Lexer -> Token stream $\downarrow$ `next_token()/tokenize()`

Lexer maintains a cursor into source, plus line/column for diagnostics. Each token's location records the start of its lexeme (filename:line:column).

- **kind** – TokenKind enum (keyword, operator, literal, etc.)
- **lexeme** – exact substring from the source (e.g., “factorial”, “<=“)
- **location** – SourceLocation
- **int_value** – optional; set for IntLiteral and boolean keywords

III. CONSTRUCTING THE LEXER

```
#include "minilang/lexer/lexer.hpp"
std::string source = /* file contents */;
miniLang::Lexer lexer(source, "path/to/file.minilang");
```

The second argument is the filename used in *SourceLocation* (defaults to "<input>").

Downstream phases(parser) should prefer `next_token()` for streaming; `tokenize()` is convenient for debugging and tests.

`const char* token_kind_name(Tokenfind kind)` maps to stable names (e.g. KW_INT, EQEQ) for logging.

Method	Behavior
<code>Token next_token()</code>	Returns one token and advances the cursor
<code>std::vector<Token> tokenize()</code>	Collects tokens until EOF or the first invalid token
<code>bool has_error()</code>	True if any error token was produced

TABLE II: Scanning Tokens

IV. COMMAND-LINE DRIVER

```
./build/minilang_lexer
tests/programs/hello.minilang
```

A. Example Output

```
KW_INT 'int' @ hello.minilang:1:1
INT '10' value=10 @ hello.minilang:2:13
EOF @ hello.minilang:13:1
```

Exit	Meaning
0	Success
1	Usage error or <code>l0exception</code>
2	Lexical error (<code>had_error()</code>)

TABLE III: Example output Meaning

V. LEXICAL GRAMMAR

Ignored output

- **Whitespace:** space, tab, carriage return, newline
- **Line comments:** `//` to end of line (not tokenized)
- **Block comments:** `/*...*/` (not tokenized)

Category	Rules
Identifier	<code>[A-Za-z_][A-Za-z0-9_]*</code> → keyword if spelling matches table, else IntLiteral
Integer	<code>[0-9]+</code> → IntLiteral; value stored in <code>int_value</code> (<code>std::stoll</code>)
Keywords	<code>int, bool, void, if, else, while, return, true, false</code>
Operators	<code>+, -, *, /, %, =, ==, !=, <, <=, >, >=, &&</code>
Punctuation	<code>(, {, }, [,], :, ,</code>

TABLE IV: Tokens

Multi-charater operators use **maximal munch** (e.g. `<=` is one token, not `<` and `=`)

Special cases

- `/` - `//` or `/*` starts a comment; otherwise *Slash* (division)
- `/` - only `||` are valid; a lone `or` or `|` is Invalid
- String literals — `"..."` is scanned but reported as Invalid (“not supported yet”); full string support is planned.

Boolean keywords `true` and `false` include `int_value` of 1 and 0 respectively.

Group	Kinds
End/ error	Eof, Invalid
Literals/names	Identifier, IntLiteral
Keywords	KwInt, KwBool, KwVoid, KwIf, KwElse, KwWhile, KwReturn, KwTrue, KwFalse
Operators	Plus, Minus, Star, Slash, Percent, Assign, EqEq, NotEq, Lt, Le, Gt, Ge, AndAnd, OrOr, Bang
Delimiters	LParen, RParen, LBrace, RBrace, LBracket, RBracket, Semicolon, Comma

TABLE V: Tokens kinds Summary

VI. ERROR HANDLING

- Errors are represented as tokens with kind `Invalid`. The lexeme holds a short diagnostic message (e.g. "unexpected character '?'", "unterminated block comment").
- Calling `error_token()` sets `had_error_ = true`. `tokenize()` stops after the first invalid token but still returns it in the token vector so drivers can print the failure site.
- Integer literals that overflow `int64_t` during parsing produce an `Invalid` token with message "integer literal out of range".

VII. NEXT STAGE

The parser will consume the token stream (via `next_token()`), build an AST, and rely on `Token::location` for syntax error reporting. The lexer API is intentionally stable for that integration.

VIII. CONCLUSION

The lexer successfully converts MiniLang source into a located token stream and establishes a clear base for the rest of the compiler. Future work will build parsing, semantics, LLVM codegen, and ML-guided optimizations on top of this front end.